

Applied Cryptography – Midterm Lab Exam (Weeks 1–4)

Task 1 – AES Encryption

Task 1B: Decrypt secret.enc

```
midterm : bash — Konsole
File Edit View Bookmarks Settings Help
[parrot@dell]~
└─ $cd midterm/
[parrot@dell]~/midterm
└─ $ls
[parrot@dell]~/midterm
└─ $echo "This file contains top secret information." > secret.txt
[parrot@dell]~/midterm
└─ $cat secret.txt
This file contains top secret information.
[parrot@dell]~/midterm
└─ $openssl enc -aes-128-cbc -in secret.txt -out secret.enc -pbkdf2 -pass pass:VeryStrongPassw0rd
[parrot@dell]~/midterm
└─ $ls
secret.enc secret.txt
[parrot@dell]~/midterm
└─ $cat secret.enc
Salted__+F000K0
          k5000Ze00a0004N00\000G!00D00:0
          a02Mw0└─[parrot@dell]~/midterm
└─ $openssl enc -d -aes-128-cbc -in secret.enc -out secret.dec -pbkdf2 -pass pass:VeryStrongPassw0rd
[parrot@dell]~/midterm
└─ $ls
secret.dec secret.enc secret.txt
[parrot@dell]~/midterm
└─ $cat secret.dec
This file contains top secret information.
[parrot@dell]~/midterm
└─ $cat secret.txt
This file contains top secret information.
[parrot@dell]~/midterm
└─ $
```

Task 2 - ECC Signature Verification

1 - generate a private key for a curve

openssl ecparam -name prime256v1 -genkey -noout -out private-key.pem

2 - generate corresponding public key

openssl ec -in private-key.pem -pubout -out public-key.pem

```
File Edit View Bookmarks Settings Help
ecc : bash — Konsole

[parrot@dell]~/midterm/ecc
$ openssl ecparam -name prime256v1 -genkey -noout -out private-key.pem
[parrot@dell]~/midterm/ecc
$ openssl ec -in private-key.pem -pubout -out public-key.pem
read EC key
writing EC key
[parrot@dell]~/midterm/ecc
$ ls
private-key.pem  public-key.pem
[parrot@dell]~/midterm/ecc
$ cat private-key.pem
-----BEGIN EC PRIVATE KEY-----
MHcCAQEEIFbPDEDQ4LKMUN8DusYy0tJBqCfvMcKZ9/S7p1goCbmoAoGCCqGSM49
AwEHoUQDQgAEuw5yXKjU0A7rTqcacR8jd0NKBQHFxaIDZavl/uxQhgnIOVESSrIq
VoIFoeX05cw41CvzgBM1U0jSooXlPre0HQ==
-----END EC PRIVATE KEY-----
[parrot@dell]~/midterm/ecc
$ cat public-key.pem
-----BEGIN PUBLIC KEY-----
MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAEuw5yXKjU0A7rTqcacR8jd0NKBQHF
xaIDZavl/uxQhgnIOVESSrIqVoIFoeX05cw41CvzgBM1U0jSooXlPre0HQ==
-----END PUBLIC KEY-----
[parrot@dell]~/midterm/ecc
$
```

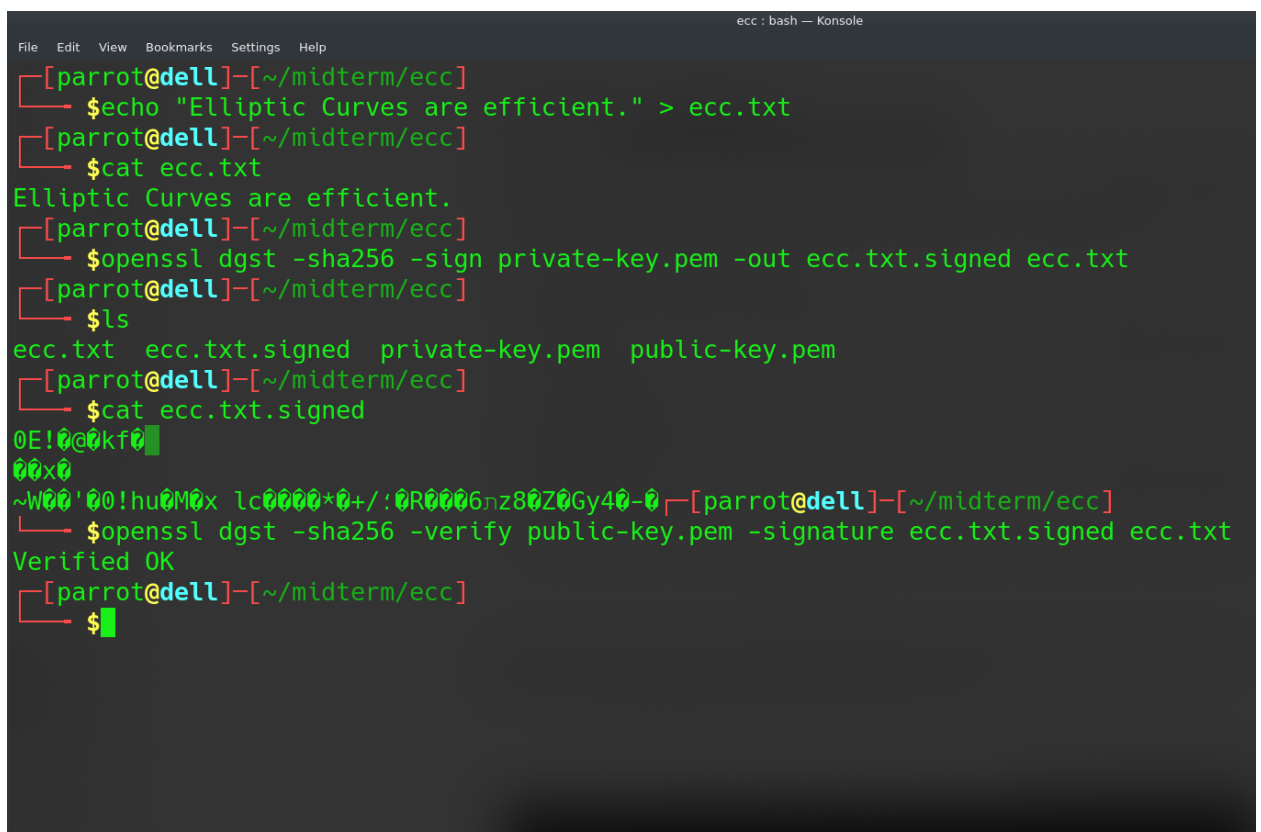
Task 2B: Sign and verify a message

3 - Let's create digital signature for ecc.txt using private-key.pem

```
openssl dgst -sha256 -sign private-key.pem -out ecc.txt.signed  
ecc.txt
```

4 - And then verify the signature using the public-key.pem. If process will be successful, we'll receive "Verified OK"

```
openssl dgst -sha256 -verify public-key.pem -signature  
ecc.txt.signed ecc.txt
```



```
ecc : bash — Konsole
File Edit View Bookmarks Settings Help
[parrot@dell]~/midterm/ecc
$ echo "Elliptic Curves are efficient." > ecc.txt
[parrot@dell]~/midterm/ecc
$ cat ecc.txt
Elliptic Curves are efficient.
[parrot@dell]~/midterm/ecc
$ openssl dgst -sha256 -sign private-key.pem -out ecc.txt.signed ecc.txt
[parrot@dell]~/midterm/ecc
$ ls
ecc.txt ecc.txt.signed private-key.pem public-key.pem
[parrot@dell]~/midterm/ecc
$ cat ecc.txt.signed
0E!0@0kf0
00x0
~W00'00!hu0M0x lc0000*0+/:0R0006nz80Z0Gy40-0
[parrot@dell]~/midterm/ecc
$ openssl dgst -sha256 -verify public-key.pem -signature ecc.txt.signed ecc.txt
Verified OK
[parrot@dell]~/midterm/ecc
$
```

Task 3 – Hashing & HMAC

1. Create data.txt with:

Never trust, always verify.

2. Hash it using Python or CLI.

3. Submit hash output and code/command.

```
hmac : bash — Konsole
File Edit View Bookmarks Settings Help
[parrot@dell]~/midterm/hmac
$ echo "Never trust, always verify." > data.txt
[parrot@dell]~/midterm/hmac
$ sha256sum data.txt
92d4c75775e8fdb68c2040b4c1ef9d3351dec5d681dba8dcb9e16f5248021cd5 data.txt
[parrot@dell]~/midterm/hmac
$
```

Task 3B: HMAC using SHA-256

1. Use the key: secretkey123

2. Create an HMAC for data.txt.

```
[parrot@dell]~/midterm/hmac
$ openssl dgst -sha256 -hmac "secretkey123" data.txt | tee hmac.data.txt
HMAC-SHA256(data.txt)= 4bc22a76203fe6f8c30832dbb5c2fcedf2ed5883859db4b433972ca4843793d9
[parrot@dell]~/midterm/hmac
$ cat hmac.data.txt
HMAC-SHA256(data.txt)= 4bc22a76203fe6f8c30832dbb5c2fcedf2ed5883859db4b433972ca4843793d9
[parrot@dell]~/midterm/hmac
$
```

Task 3C: Integrity Check

1. Change one letter in data.txt.

(Changed upper-case “N” with lowercase “n”)

2. Recompute HMAC.

```
File Edit View Bookmarks Settings Help
[parrot@dell]~/midterm/hmac
└─$ nano data.txt
[parrot@dell]~/midterm/hmac
└─$ cat data.txt
never trust, always verify.
[parrot@dell]~/midterm/hmac
└─$ openssl dgst -sha256 -hmac "secretkey123" data.txt | tee changed-hmac.data.txt
HMAC-SHA256(data.txt)= 61c780483572f1e949b8ec71857f3e3f3935397711781e23ce3c4417e7a315d6
[parrot@dell]~/midterm/hmac
└─$
[parrot@dell]~/midterm/hmac
└─$ ls
changed-hmac.data.txt  data.txt  hmac.data.txt
[parrot@dell]~/midterm/hmac
└─$ cat hmac.data.txt
HMAC-SHA256(data.txt)= 4bc22a76203fe6f8c30832dbb5c2fcedf2ed5883859db4b433972ca4843793d9
[parrot@dell]~/midterm/hmac
└─$ cat changed-hmac.data.txt
HMAC-SHA256(data.txt)= 61c780483572f1e949b8ec71857f3e3f3935397711781e23ce3c4417e7a315d6
[parrot@dell]~/midterm/hmac
└─$
```

3. Explain what happens and why HMAC is important.

When we compute an HMAC (Hash-based Message Authentication Code), we are generating a cryptographic hash of your data combined with a secret key. This ensures both data integrity and authenticity.

When we later modified a single letter in the file from "Never" to "never" and recomputed the HMAC, the output was completely different. This dramatic change demonstrates a fundamental property of cryptographic hash functions: the avalanche effect, where even a tiny input change results in a vastly different output.

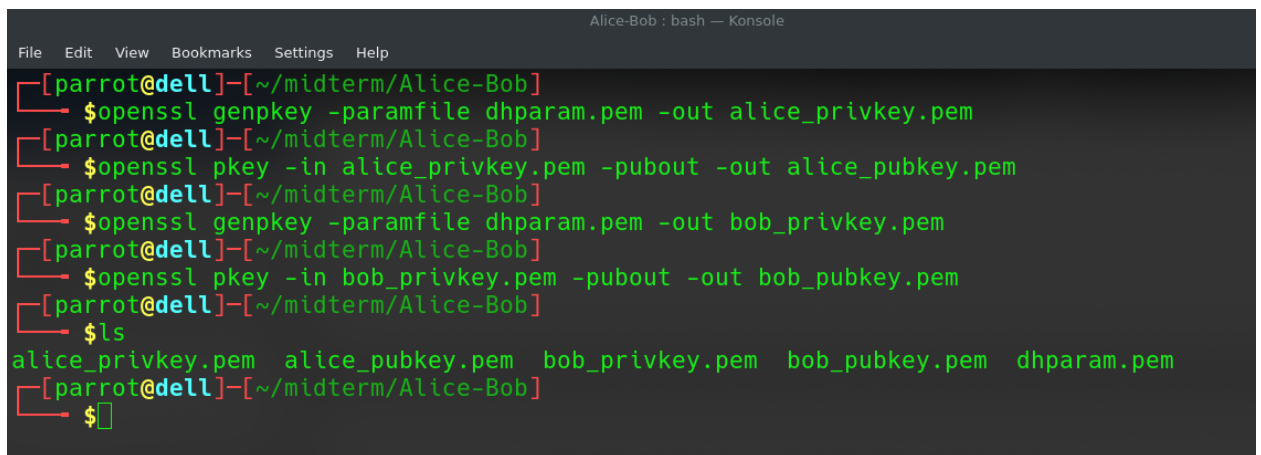
HMAC is crucial in secure communications because it allows the recipient to verify that the message was not tampered with and that it was generated by someone who knows the shared secret key. Unlike plain hashes (like SHA-256 alone), HMACs prevent attackers from simply replacing both the file and its hash. Without the key, they cannot forge a valid HMAC.

In real-world applications, HMACs are used in APIs, secure file transfers, and communication protocols like HTTPS or TLS. They provide a lightweight yet powerful mechanism to ensure that the data has not been altered in transit and is genuinely from the expected source.

[illegible]

2) Generate Private and Public Keys for Alice and for Bob

```
openssl genpkey -paramfile dhparam.pem -out alice_privkey.pem  
openssl pkey -in alice_privkey.pem -pubout -out alice_pubkey.pem  
openssl genpkey -paramfile dhparam.pem -out bob_privkey.pem  
openssl pkey -in bob_privkey.pem -pubout -out bob_pubkey.pem
```



```
Alice-Bob : bash — Konsole  
File Edit View Bookmarks Settings Help  
[parrot@dell]~/midterm/Alice-Bob  
$ openssl genpkey -paramfile dhparam.pem -out alice_privkey.pem  
[parrot@dell]~/midterm/Alice-Bob  
$ openssl pkey -in alice_privkey.pem -pubout -out alice_pubkey.pem  
[parrot@dell]~/midterm/Alice-Bob  
$ openssl genpkey -paramfile dhparam.pem -out bob_privkey.pem  
[parrot@dell]~/midterm/Alice-Bob  
$ openssl pkey -in bob_privkey.pem -pubout -out bob_pubkey.pem  
[parrot@dell]~/midterm/Alice-Bob  
$ ls  
alice_privkey.pem  alice_pubkey.pem  bob_privkey.pem  bob_pubkey.pem  dhparam.pem  
[parrot@dell]~/midterm/Alice-Bob  
$
```

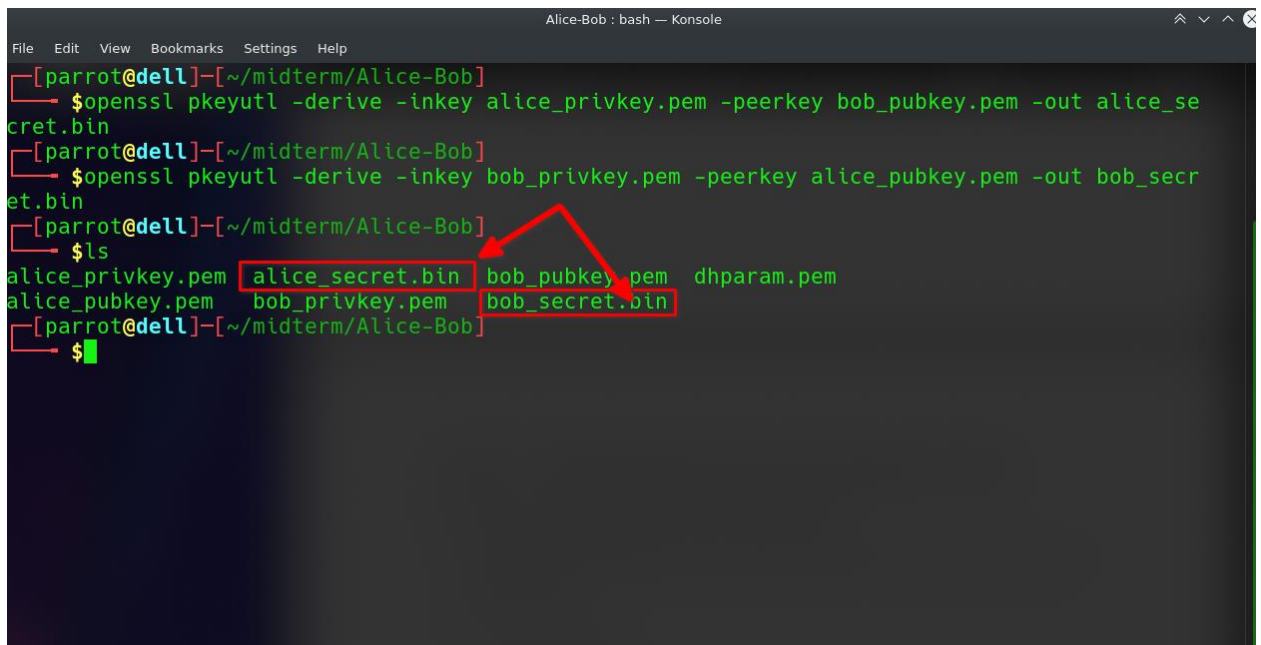

3) Derive Shared Secrets

Alice derives shared secret (using her privkey and Bob's pubkey):

```
openssl pkeyutl -derive -inkey alice_privkey.pem -peerkey bob_pubkey.pem -out alice_secret.bin
```

Bob derives shared secret (using his privkey and Alice's pubkey):

```
openssl pkeyutl -derive -inkey bob_privkey.pem -peerkey alice_pubkey.pem -out bob_secret.bin
```



The screenshot shows a terminal window titled "Alice-Bob : bash — Konsole". The user, identified as "parrot@dell", is in the directory "~/midterm/Alice-Bob". The terminal shows the following commands and output:

```
[parrot@dell]~/midterm/Alice-Bob
$ openssl pkeyutl -derive -inkey alice_privkey.pem -peerkey bob_pubkey.pem -out alice_secret.bin
[parrot@dell]~/midterm/Alice-Bob
$ openssl pkeyutl -derive -inkey bob_privkey.pem -peerkey alice_pubkey.pem -out bob_secret.bin
[parrot@dell]~/midterm/Alice-Bob
$ ls
alice_privkey.pem  alice_secret.bin  bob_pubkey.pem  dhparam.pem
alice_pubkey.pem  bob_privkey.pem  bob_secret.bin
```

Red boxes highlight the files `alice_secret.bin` and `bob_secret.bin` in the `ls` output. A red arrow points from the `alice_secret.bin` box to the `bob_secret.bin` box, indicating that both parties have derived the same shared secret.

4) Check file contents

cat alice_pubkey.pem

cat bob_pubkey.pem

xxd alice_secret.bin

xxd bob_secret.bin

```
Alice-Bob : bash — Konsole
File Edit View Bookmarks Settings Help
[parrot@de11]~/midterm/Alice-Bob
$ls
alice_privkey.pem  alice_secret.bin  bob_pubkey.pem  dhparam.pem
alice_pubkey.pem  bob_privkey.pem  bob_secret.bin
[parrot@de11]~/midterm/Alice-Bob
$cat alice_pubkey.pem
-----BEGIN PUBLIC KEY-----
MIICJDCCARcGCSqGSIb3DQEDATCCAQgCggEBAMMSCni+FwUpYxRLmo+K0Hyfzc7W
iWRVcSKMZ5QrUzZPsHrILkCseRmzixiYdFGS6sX74Xz1r7TXsYJvrSS8w4NCJ88+
p760+YeHoB1AfBT01+LsCytnjqFyBFdy1G/Kykc2lF2rlPB0f7WVhhSly/v0tBWc
BZkmf3WjKo+SgQNFZtl6fn8B5sekQ1nTJT6yJv0qwq3uDcsjFFmmN63jccEhxCBG
oE6bqertK1fTLHXdTBIQTqywbZrLqyaQNeCkN8ux3CmI369CGgXCDT7PXkNUPDbG
8RevjPXtJy6WTIUhot1wpaIrm5QPq24KpJ38vIGsPymwDXFlb1St62r00rMCAQID
ggEFAAKCAQATg1qaNiAavgFEmfiziwMFy+yD8PXgW8sJXXV/Bf206vx8azSYBJW9
2zEi9EYz0XrvckA4Lf2BMwkk2su7qZrT1daM1lVC/raLEr2iHjXaQuyyq0Eryz6I
45gzj5qdMydgZJoqTuZiEysNUmvCeD6efX6WTI8QGMGUBfe+5idIu+yH1kXxz6mb
XRvtMdbrywfnl0LEiNfxz7CCNdwnuGmIN98I9FcAy9sxE3TP0TMctjSH/+P5Ri4l
XWdQAh5b4US0vCRnc0/kxeVP7bjmy/jXPU5NP9tBPzDUYca1M1ij9K/8Qyr1urII
GH+ypCuYzirleWovzvkvtpCjSH0ryIgG
-----END PUBLIC KEY-----
[parrot@de11]~/midterm/Alice-Bob
$cat bob_pubkey.pem
-----BEGIN PUBLIC KEY-----
MIICJDCCARcGCSqGSIb3DQEDATCCAQgCggEBAMMSCni+FwUpYxRLmo+K0Hyfzc7W
iWRVcSKMZ5QrUzZPsHrILkCseRmzixiYdFGS6sX74Xz1r7TXsYJvrSS8w4NCJ88+
p760+YeHoB1AfBT01+LsCytnjqFyBFdy1G/Kykc2lF2rlPB0f7WVhhSly/v0tBWc
BZkmf3WjKo+SgQNFZtl6fn8B5sekQ1nTJT6yJv0qwq3uDcsjFFmmN63jccEhxCBG
oE6bqertK1fTLHXdTBIQTqywbZrLqyaQNeCkN8ux3CmI369CGgXCDT7PXkNUPDbG
8RevjPXtJy6WTIUhot1wpaIrm5QPq24KpJ38vIGsPymwDXFlb1St62r00rMCAQID
ggEFAAKCAQB4fCVQf89EDCRXzFQUdX8crQY/HX9Z/hjD3xDSLJqv0tmqXGzGnnzA
oQdW8YGmU6LbW63ZGrs3+JQf4dRniPh/dVkhJId9rVXYH1xGzA+ZEFh4hvsEoNDs
Pb8FCR70JYLBW0vyex0MvhEq31QmTUqgrqbg3ovFa9tC8uKum2KQxNPqxRktFY2z
VWYYpcavW/GgBwhnH7rE5EgHD/fsdTVF5lnbgWRzYLnGV92lhfCMdk+0cMTXUcGe
iLVFuBcALAFroqsQDNdLscc0WF38KdxcMBq79M8mlmdqUkLkZ8CG8wGg6vGkeQd6
sM/Eom2jUpU2wfkrm0EVBksJT0yKX320
-----END PUBLIC KEY-----
[parrot@de11]~/midterm/Alice-Bob
$
```

```
Alice-Bob : bash — Konsole
File Edit View Bookmarks Settings Help
[parrot@dell]~[~/midterm/Alice-Bob]
$xxd alice_secret.bin
00000000: 1b11 185b dfff 6bad 535b 2de0 7125 d32f ...[.k.S[-.q%./
00000010: 2a14 50fa c868 b3b6 b13d ec78 9623 966c *.P..h...=.x.#.l
00000020: 64ac a2af 68c6 f180 108f 2e58 1712 c175 d...h.....X...u
00000030: e442 89a1 c44b 5524 511f 3b9f 6f5f 9aac .B...KU$Q.;.o_..
00000040: 6aee 1c5c 5dd9 22e3 13d3 fdd9 a382 60e0 j...\]."......\
00000050: ee5c 33b2 b14c 2cfe 640f 20c1 4f76 652f .\3..L,.d..0ve/
00000060: 0786 82c4 073c 8e81 301c 4386 eb53 3932 .....<..0.C..S92
00000070: a421 52a5 238f 6143 04eb ecfa dd81 f3a4 ..!R.#.aC.....
00000080: 14fb 9144 8979 0321 eeef f617 ca4a 7426 ...D.y.!.....Jt&
00000090: fb95 cbaa 65b7 30c2 7ca4 4790 8d04 394e ....e.0.|.G...9N
000000a0: 3f14 8383 a132 071b b3bb 9f8f 1743 5720 ?....2.....CW
000000b0: 5765 61bb 945a 9509 65f0 1798 153f 585e Wea..Z..e....?X^
000000c0: 6ff7 9892 e593 fa43 bbbf 2076 d8ab b379 o.....C.. v...y
000000d0: dfaa 753c 094d 8f44 4e3f 62f3 7a01 1f68 ..u<.M.DN?b.z..h
000000e0: 47a3 21b2 d856 1bac fc17 16e1 c1dc 0dd0 G.!..V.....
000000f0: d467 26d7 3e9b 12aa aeee 450f 32c1 7f34 .g&.>.....E.2..4
[parrot@dell]~[~/midterm/Alice-Bob]
$xxd bob_secret.bin
00000000: 1b11 185b dfff 6bad 535b 2de0 7125 d32f ...[.k.S[-.q%./
00000010: 2a14 50fa c868 b3b6 b13d ec78 9623 966c *.P..h...=.x.#.l
00000020: 64ac a2af 68c6 f180 108f 2e58 1712 c175 d...h.....X...u
00000030: e442 89a1 c44b 5524 511f 3b9f 6f5f 9aac .B...KU$Q.;.o_..
00000040: 6aee 1c5c 5dd9 22e3 13d3 fdd9 a382 60e0 j...\]."......\
00000050: ee5c 33b2 b14c 2cfe 640f 20c1 4f76 652f .\3..L,.d..0ve/
00000060: 0786 82c4 073c 8e81 301c 4386 eb53 3932 .....<..0.C..S92
00000070: a421 52a5 238f 6143 04eb ecfa dd81 f3a4 ..!R.#.aC.....
00000080: 14fb 9144 8979 0321 eeef f617 ca4a 7426 ...D.y.!.....Jt&
00000090: fb95 cbaa 65b7 30c2 7ca4 4790 8d04 394e ....e.0.|.G...9N
000000a0: 3f14 8383 a132 071b b3bb 9f8f 1743 5720 ?....2.....CW
000000b0: 5765 61bb 945a 9509 65f0 1798 153f 585e Wea..Z..e....?X^
000000c0: 6ff7 9892 e593 fa43 bbbf 2076 d8ab b379 o.....C.. v...y
000000d0: dfaa 753c 094d 8f44 4e3f 62f3 7a01 1f68 ..u<.M.DN?b.z..h
000000e0: 47a3 21b2 d856 1bac fc17 16e1 c1dc 0dd0 G.!..V.....
000000f0: d467 26d7 3e9b 12aa aeee 450f 32c1 7f34 .g&.>.....E.2..4
[parrot@dell]~[~/midterm/Alice-Bob]
$
```

As we can see in the screenshot above, Alice and Bob's shared secret key contents are identical. They are now ready to continue exchanging information with each other using this shared key.

Task 4B: Real-Life Application

Diffie-Hellman (DH) is a foundational cryptographic algorithm used widely in secure communication systems to establish a shared secret between two parties over an insecure channel. One of its most common applications is in the TLS (Transport Layer Security) handshake, which secures web traffic. During a TLS handshake, ephemeral Diffie-Hellman (DHE) or elliptic-curve Diffie-Hellman (ECDHE) is used to create a temporary shared key between a client and server. This ensures forward secrecy, meaning even if the server's private key is compromised in the future, past communications remain secure. Another major use case is in secure messaging applications, such as Signal and WhatsApp, which implement the Signal Protocol. This protocol uses a variation called the Double Ratchet algorithm, which includes DH exchanges to generate and frequently refresh encryption keys. This ensures not only forward secrecy but also post-compromise security, meaning even if a device is compromised, future messages can still remain private.

The importance of Diffie-Hellman in secure communication lies in its ability to enable confidential key exchange without transmitting the actual key over the network. Unlike symmetric encryption, which requires both parties to already share a secret, DH allows secure key agreement even when the two sides have never communicated before. This makes it a cornerstone of modern cryptographic systems. Without such a method, all secure internet traffic, from online banking to private chats, would be vulnerable to eavesdropping and interception. By facilitating the safe exchange of encryption keys, Diffie-Hellman helps ensure data confidentiality, integrity, and trust in digital communication.

<https://github.com/quazary/midterm.git>